Synchronization practice

Progress & feedback report

Student: Xiangbin Luo

Section / identifier: 250

Report generated: 10/21/2026, 10:58:17 AM

Started: 10/21/2026, 10:46:07 AM

Approximate visible-tab time: 12 minutes. Timing is informational, not a grade.

SUMMARY

4 / 6 exercises completed

1 / 6 concept questions correct on first attempt

1 / 6 completed without a guided solution

1. Count every button press

Status: Completed | Concept attempts: 1 | Placement checks: 3 | Guided solution used

2. Reserve the last seat

Status: Skipped / not attempted | Concept attempts: 0 | Placement checks: 0 | Not completed

3. Read a consistent sample

Status: Skipped / not attempted | Concept attempts: 0 | Placement checks: 0 | Not completed

4. Hand off one sample buffer

Status: Completed | Concept attempts: 3 | Placement checks: 2 | Completed independently

5. Keep every event in a burst

Status: Completed | Concept attempts: 3 | Placement checks: 5 | Guided solution used

6. Transfer a complete measurement

Status: Completed | Concept attempts: 2 | Placement checks: 3 | Guided solution used

WHAT THIS ACTIVITY PRACTICED

Mutexes protect a whole shared operation. Binary semaphores pass turns. Counting semaphores retain event or resource tokens. Queues transfer copied data items.

SUBMISSION

Download this PDF and submit it through the course submission page. The website does not submit automatically. The following pages record first responses, attempts, final placements, and corrective feedback.

Practice record generated in the student browser. This is a formative learning activity, not a server-verified examination score.

1. Count every button press

Xiangbin Luo · MUTEX

CONCEPT QUESTION

Both tasks read events = 7, add 1, and store 8. What went wrong?

First answer: The read and write were not protected as one operation.

First attempt: Correct | Attempts: 1

Correct answer: The read and write were not protected as one operation.

Each increment is a read-modify-write. Two overlapping increments can overwrite each other, losing an event.

PLACEMENT FEEDBACK

Checks: 3 | Hints requested: 1 | Guided solution: yes

First check: Needs revision. Do not hold this mutex while waiting for the next external request or button event. That can block the other task indefinitely. Take before reading events; give only after writing the new value. Leave each blocking wait outside the lock.

2 unsuccessful placement check(s) before completion.

FINAL PLACEMENT / YELLOW MARKS INSERTED LINES

task_a:

Before: unsigned n = events;

```
xSemaphoreTake(count_lock, portMAX_DELAY);
```

After: events = n + 1;

```
xSemaphoreGive(count_lock);
```

task_b:

Before: unsigned n = events;

```
xSemaphoreTake(count_lock, portMAX_DELAY);
```

After: events = n + 1;

```
xSemaphoreGive(count_lock);
```

TAKEAWAY

Both tasks protect the same read-modify-write with count_lock. The button wait stays outside, allowing the other task to continue.

2. Reserve the last seat

Xiangbin Luo · MUTEX

STATUS

Skipped / not attempted

This exercise is not counted as completed.

CONCEPT QUESTION

Which operations must stay together inside the mutex?

No concept answer was submitted.

PLACEMENT PROGRESS

Placement checks: 0 | Hints: 0 | Guided solution: no

No code placement saved.

Return to this exercise using Jump to whenever you want to continue. Skipping does not count as a correct answer or a completion.

3. Read a consistent sample

Xiangbin Luo · MUTEX

STATUS

Skipped / not attempted

This exercise is not counted as completed.

CONCEPT QUESTION

Is protecting only the writer sufficient?

No concept answer was submitted.

PLACEMENT PROGRESS

Placement checks: 0 | Hints: 0 | Guided solution: no
No code placement saved.

Return to this exercise using Jump to whenever you want to continue. Skipping does not count as a correct answer or a completion.

4. Hand off one sample buffer

Xiangbin Luo · BINARY

CONCEPT QUESTION

Which semaphore should receive the one initial token?

First answer: full: the consumer should run first.

First attempt: Incorrect | Attempts: 3

Correct answer: empty: the producer may fill the unused buffer.

Both semaphores start empty. Seed empty so A can produce first; full must wait for a complete buffer.

PLACEMENT FEEDBACK

Checks: 2 | Hints requested: 0 | Guided solution: no

First check: Needs revision. A takes empty, fills, then gives full. B takes full, processes, then gives empty. Seed empty before starting either task.

1 unsuccessful placement check(s) before completion.

FINAL PLACEMENT / YELLOW MARKS INSERTED LINES

In app_main(), before starting either task:

```
xSemaphoreGive(empty);
```

task_a:

Before: fill_samples(samples, 4);

```
xSemaphoreTake(empty, portMAX_DELAY);
```

After: fill_samples(samples, 4);

```
xSemaphoreGive(full);
```

task_b:

Before: process_samples(samples, 4);

```
xSemaphoreTake(full, portMAX_DELAY);
```

After: process_samples(samples, 4);

```
xSemaphoreGive(empty);
```

TAKEAWAY

Ownership alternates. The producer cannot overwrite the buffer while B is reading. process_samples completes before returning and retains no pointer.

5. Keep every event in a burst

Xiangbin Luo · COUNTING

CONCEPT QUESTION

Which creation call preserves all four events if A finishes before B starts?

First answer: pending = xSemaphoreCreateBinary();

First attempt: Incorrect | Attempts: 3

Correct answer: pending = xSemaphoreCreateCounting(4, 0);

Start at zero because no events have happened yet. A maximum of four can retain the complete burst.

PLACEMENT FEEDBACK

Checks: 5 | Hints requested: 1 | Guided solution: yes

First check: Needs revision. Creation is before both tasks start. A gives once per iteration; B takes before each action. Neither task should give and immediately take its own token.

4 unsuccessful placement check(s) before completion.

FINAL PLACEMENT / YELLOW MARKS INSERTED LINES

In app_main(), before starting either task:

```
pending = xSemaphoreCreateCounting(4, 0);
```

task_a:

After: // One event occurs in this iteration.

```
xSemaphoreGive(pending);
```

task_b:

Before: handle_one_event();

```
xSemaphoreTake(pending, portMAX_DELAY);
```

TAKEAWAY

The count represents pending work, not stored data. Capacity four prevents overflow for this finite burst. B waits after consuming all four tokens.

6. Transfer a complete measurement

Xiangbin Luo · QUEUE

CONCEPT QUESTION

Why can A reuse its local m after xQueueSend returns successfully?

First answer: The queue keeps A paused until B finishes.

First attempt: Incorrect | Attempts: 2

Correct answer: The queue has copied the complete Measurement value.

With sizeof(Measurement), the whole structure is copied. Queuing a pointer would not copy the pointed-to structure.

PLACEMENT FEEDBACK

Checks: 3 | Hints requested: 1 | Guided solution: yes

First check: Needs revision. Create before starting tasks. A sends after m exists. B receives before printing. Use &m to pass the address for a value copy.

2 unsuccessful placement check(s) before completion.

FINAL PLACEMENT / YELLOW MARKS INSERTED LINES

In app_main(), before starting either task:

```
measurements = xQueueCreate(3, sizeof(Measurement));
```

task_a:

Before: vTaskDelay(pdMS_TO_TICKS(100));

```
xQueueSend(measurements, &m, portMAX_DELAY);
```

task_b:

Before: printf("%u: %d\n", m.sequence, m.light);

```
xQueueReceive(measurements, &m, portMAX_DELAY);
```

TAKEAWAY

Each FIFO item keeps sequence and light together. A blocks if full; B blocks if empty. No extra application mutex is needed.